

ADVANCE PROBLEM PROGRAMMING

SLO 1: Understand Concurrency

1. What is a Thread in Java? How Does It Differ from a Process?

A **thread** is a lightweight unit of execution within a Java program. A Java application can have multiple threads running concurrently.

A **process** is an independent program that is executing in the operating system. A process has its own memory space, while threads within the same process share memory.

Feature	Process	Thread
Definition	Independent executing program	Smallest unit of execution within a process
Memory	Has separate memory space	Shares memory with other threads
Creation	More resource-intensive	Less resource-intensive
Communication	Requires inter-process communication	Can communicate through shared memory
Example	Running a Java application	A thread performing a calculation inside the application

Example of creating a thread:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();
```

```
t.start();  
}  
}
```

Output:

Thread is running

2. Identify Concurrency Issues in Shared Memory

When multiple threads access and modify the same shared data at the same time, concurrency problems can occur.

The major issues are:

- 1. Race Condition:**

Occurs when multiple threads access shared data simultaneously and the final result depends on the order of execution.

- 2. Data Inconsistency:**

Shared data may contain incorrect or unexpected values because multiple threads modify it concurrently.

- 3. Deadlock:**

Occurs when two or more threads wait indefinitely for resources held by each other.

- 4. Starvation:**

Occurs when a thread is continuously denied access to the resources it needs.

- 5. Synchronization Problems:**

Without proper synchronization, operations on shared data may not be performed safely.

Example of a race condition:

```
class Counter {  
    int count = 0;
```

```
void increment() {  
    count++;  
}  
}
```

```
public class Main {  
    public static void main(String[] args) throws Exception {  
        Counter c = new Counter();  
  
        Thread t1 = new Thread() -> {  
            for (int i = 0; i < 1000; i++)  
                c.increment();  
        });  
  
        Thread t2 = new Thread() -> {  
            for (int i = 0; i < 1000; i++)  
                c.increment();  
        });  
  
        t1.start();  
        t2.start();  
  
        t1.join();  
        t2.join();  
  
        System.out.println("Count = " + c.count);  
    }  
}
```

```
}  
  
}
```

The expected value is 2000, but without synchronization, the result may sometimes be less because of a **race condition**.

3. Define Thread Lifecycle in Java with Example

The **thread lifecycle** describes the different states through which a thread passes from creation to termination.

The main Java thread states are:

1. **NEW** – The thread is created but has not started.
2. **RUNNABLE** – The thread is ready to run or is running.
3. **BLOCKED** – The thread is waiting to acquire a lock.
4. **WAITING** – The thread is waiting indefinitely for another thread.
5. **TIMED_WAITING** – The thread waits for a specified period.
6. **TERMINATED** – The thread has completed execution.

Example:

```
class MyThread extends Thread {  
    public void run() {  
        try {  
            Thread.sleep(1000);  
            System.out.println("Thread is running");  
        } catch (InterruptedException e) {  
            System.out.println("Thread interrupted");  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) throws Exception {  
  
        MyThread t = new MyThread();  
  
        System.out.println("State after creation: " + t.getState());  
  
        t.start();  
  
        System.out.println("State after start: " + t.getState());  
  
        t.join();  
  
        System.out.println("State after completion: " + t.getState());  
    }  
}
```

Possible Output:

State after creation: NEW

State after start: RUNNABLE

Thread is running

State after completion: TERMINATED